Data Analysis and prediction on Bank market campaign

Abstract

Banks use telemarketing to expand clients and encourage them to make a term deposit. It is expensive to have billions of calls and few people subscribed to give a term deposit. A machine learning approach is useful when dealing with large data, which is useful with bank data. In this report, a decision tree classifier model in Weka Explorer marked as J48 is applied to predict whether the client will subscribe to a term deposit given 16 attributes as predictors. The main goal of this report is to go through the process of data analysis, including understanding data, data cleaning, select baseline model, error analysis, parameter tuning, and final evaluation. In conclusion, the final evaluation with the testing dataset achieved 91.04% accuracy, which is 0.1% lower than the accuracy of the original no error analysis or parameter tuned model.

Introduction

Telemarketing is a campaign strategy frequently used by the banks. It is expensive when only a few people subscribed to the plan. It is essential for banks to locate customers that are more likely to subscribe, so next time they can search for that specific population of customers and increase profit by saving time and cost. The data used in this report is created by Sérgio Moro and Paulo Cortez. It was originally used in their publication of "Using Data Mining for Bank Direct Marketing: An Application of the CRISP-DM Methodology". They also have another published paper known as "A data-driven approach to predict the success of bank telemarketing" using a similar dataset that has 150 features. The researchers were able to select

the 22 features that are most useful, and discovered the Neural Network model yields the best results at predicting whether the client will subscribe to a term deposit (S. Moro et al. 8). Since the research was based on similar data with fewer features, it is useful to learn that the data driven approach is applicable in analyzing telemarketing, and motivates me to use less features and simpler methods to develop a classification model to acquire better accuracy than my original data.

Another research titled “Multimodal Banking Dataset: Understanding Client Needs through Event Sequences” done by a group of researchers focused on creating the multimodal banking dataset. Their goal was to provide a large multi-sources bank dataset, and one of the discoveries during their research was that anonymity in client information does not affect the performance of the model (Mollaev et al. 2). This finding during their research increased the credibility of the dataset I planned to use, which is composed of anonymous data with general information of age and broad job titles. Hence, I’m more confident in applying the machine learning model on the telemarketing data.

Background and Data Cleaning

The dataset is related with direct marketing campaigns of a Portuguese banking institution summarized by Sérgio Moro and Paulo Cortez. There are a total of four dataset in the folder they provided, and the data used in this report is titled "bank-full.csv” with 45211 examples and 16 features plus the output attribute, which is whether the client subscribed to a term deposit. Here are the description of the attributes:

1. age (numeric)
2. job : type of job (categorical: "admin.", "unknown", "unemployed", "management", "Housemaid", "entrepreneur", "student", "blue-collar","self-employed", "retired", "technician", "services")
3. marital : marital status (categorical: "married", "divorced", "single")
    a. "divorced" means divorced or widowed
4. education (categorical: "unknown","secondary","primary","tertiary")
5. default: has credit in default? (binary: "yes","no")
6. balance: average yearly balance, in euros (numeric)
7. housing: has a housing loan? (binary: "yes","no")
8. loan: has a personal loan? (binary: "yes","no")

related with the last contact of the current campaign:

9. contact: contact communication type (categorical: "unknown","telephone","cellular")
10. day: last contact day of the month (numeric)
11. month: last contact month of year (categorical: "jan", "feb", "mar", ..., "nov", "dec")
12. duration: last contact duration, in seconds (numeric) other attributes:
13. campaign: number of contacts performed during this campaign and for this client (numeric, includes last contact)
14. pdays: number of days that passed by after the client was last contacted from a previous campaign
    a. (numeric, -1 means client was not previously contacted)

15. previous: number of contacts performed before this campaign and for this client (numeric)
16. poutcome: outcome of the previous marketing campaign
    a. (categorical: "unknown", "other", "failure", "success")

Output variable (desired target):

17. Y: Has the client subscribed to a term deposit? (binary: "yes","no")

The data was previously cleaned by the creator of the dataset, so there are no NA values. However, the dataset was not splitted and was provided with commas once opened in the Excel form, which can't be directly loaded into Weka data explorer. Therefore, extra data cleaning is required.

The first step was using the excel's "Text to Columns" tool to separate the attributes into different columns by commas. The second step was deciding how to properly split the data instances. Since there are 45211 instances in the dataset, it is sufficient to use 70 percent as training data; 20 percent as development data; and 10 percent as final testing data. For random selection, I applied the RAND() function in Excel to generate a column of random digits less than 1 but greater than 0. Using the sorting function in Excel by column containing the RAND() function, I now randomized the instances in my dataset. The subsets are separated based on the new randomly ordered dataset. After successfully loading all three datasets into Weka explorer, I saved each dataset into the attribute-relation file format and using text edit, I ensured all three dataset's attributes are the same order. This is important because Weka treats text variables such as "job" as nominal, so the order of the first few instances that contain all job titles will be the

default setting in attribute-relation file format. The training, testing, and development data have different instances with different order of first appeared job, and Weka can't process two different orders in the supplied test. Therefore, it is important to ensure the attribute relation in all files are identical.

The last step was choosing the class value. The dataset was created with variable y, which is whether the client subscribed to a term deposit, as the class value. Interested in the same question as previous researchers, I choose to keep the y-variable as the class value. According to the Readme.txt file provided by the creator, the data is "related with direct marketing campaigns of a Portuguese banking institution, and the marketing campaigns were based on phone calls. Often, more than one contact to the same client was required, in order to assess if the product (bank term deposit) would be (or not) subscribed" (Moro et al.). Therefore the class value comes from actual confirmation made by the client through phone calls.

With the datasets ready to use and the class value determined, the next step is to select the baseline model and perform error analysis to find out which feature is potentially removable.

Baseline selection and Error Analysis

The following section contains two parts, the first part is finding the best baseline model using the development dataset, and the second part is applying error analysis on the training data to filter out features that potentially reduce the performance or provide negligible information.

Below are the model performances in percentage.

	Development Data	Training Data
Naive Bayes	86.82	88.1
Bayes Net	88.14	88.2
Simple Logistics	90.01	90.05
Logistics	90.08	90.09
Decision Table	89.48	90.04
Tree-J48	89.47	90.2
Random Forest	90.09	90.19

Table 1: Model Performance: Development data and Training Data using different models

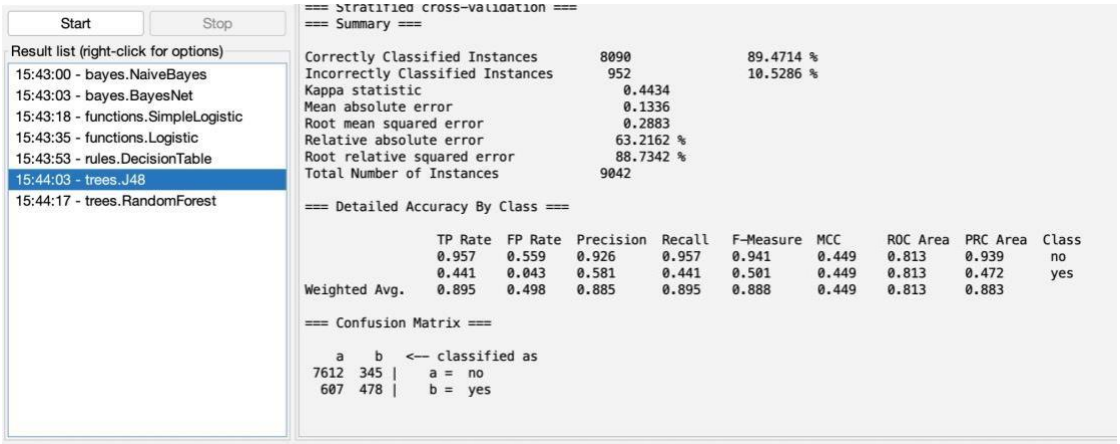

Figure 1: Development data-J48 model

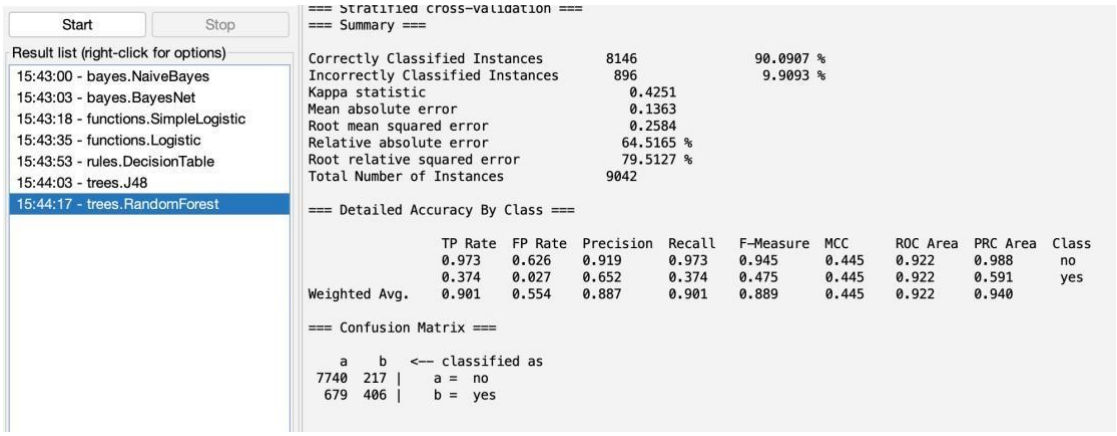

Figure 2: Development data-Random Forest model

According to the development data, Random forest has the best performance, followed by a logistic regression model. However, with detailed observations, the Tree-J48 model has a higher rate of correctly identifying clients that actually subscribed to a term deposit(478 true yes) than Random Forest (406 true yes). Since our goal is to identify clients who make deposits, being able to identify clients that actually make deposits will typically produce higher profit for the bank. It is also noted that logistic regression is typically difficult to interpret the values because it involves logarithm in the computation. Hence I ran these models again on the Training dataset,

and observed that J48 actually has a tiny higher accuracy than Random forest. The result of the performance(Figure 3) led me to choose J48 as the baseline model.

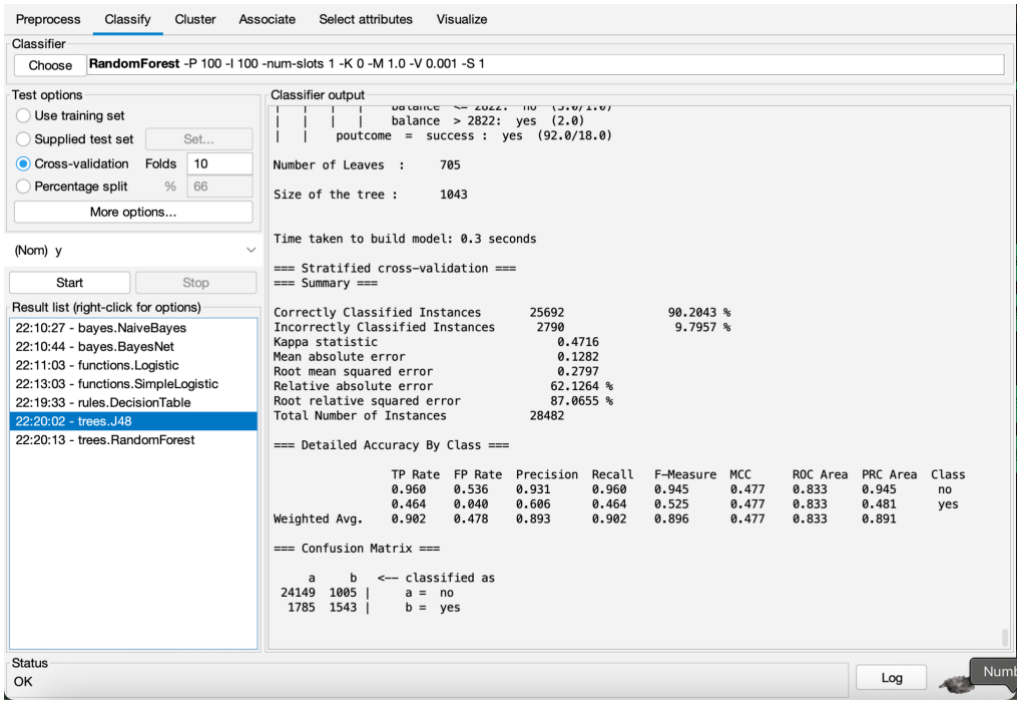

Figure 3: Performance of J48 Model on Training Data

An additional note is that SMO and Multi-layered Perceptron were attempted but they never stopped running, hence I had to forbid these two models considering time and efficiency costs. Once I determined J48 as the baseline model, the next step is running error analysis on the training data. Specifically, Vertical and Horizontal Comparison is applied.

Error Analysis – Vertical Analysis

Predicted → Actual ↓	No	Yes
No	24149	1005
Yes	1785	1543

Table 2: Confusion Matrix of J48 model using training data

In Vertical Comparison, I focused on "How a false instance is similar to the instance that was correctly labeled", specifically similarity between clients that subscribed but were classified as not subscribed. This is more important because if I can identify features that are similar and remove it or modify it then the bank can become more accurate in identifying potential subscribers. The graph I'm using to get the instances is the following picture. The Dependent variable and color are actual class values, and the Independent variable is the predicted class.

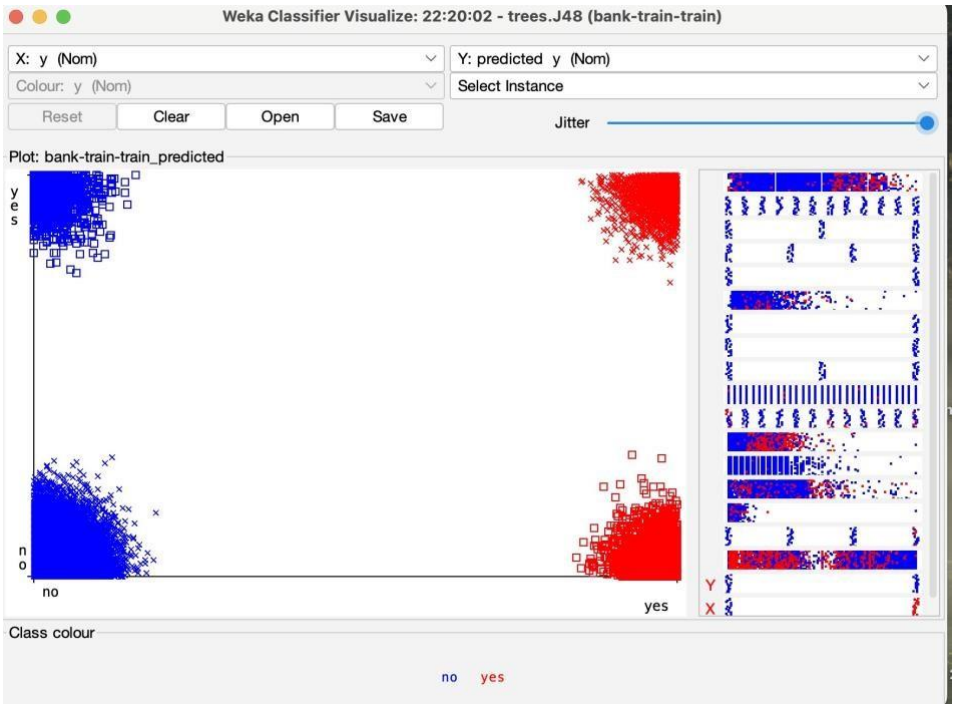

Figure 4: Weka Visualization of Observe class vs. Actual class

	Predicted No
Actual No	**24149**
Actual Yes	**1785**

Table 3: Subscribe but Classified as Not vs. Actually Didn't Subscribe

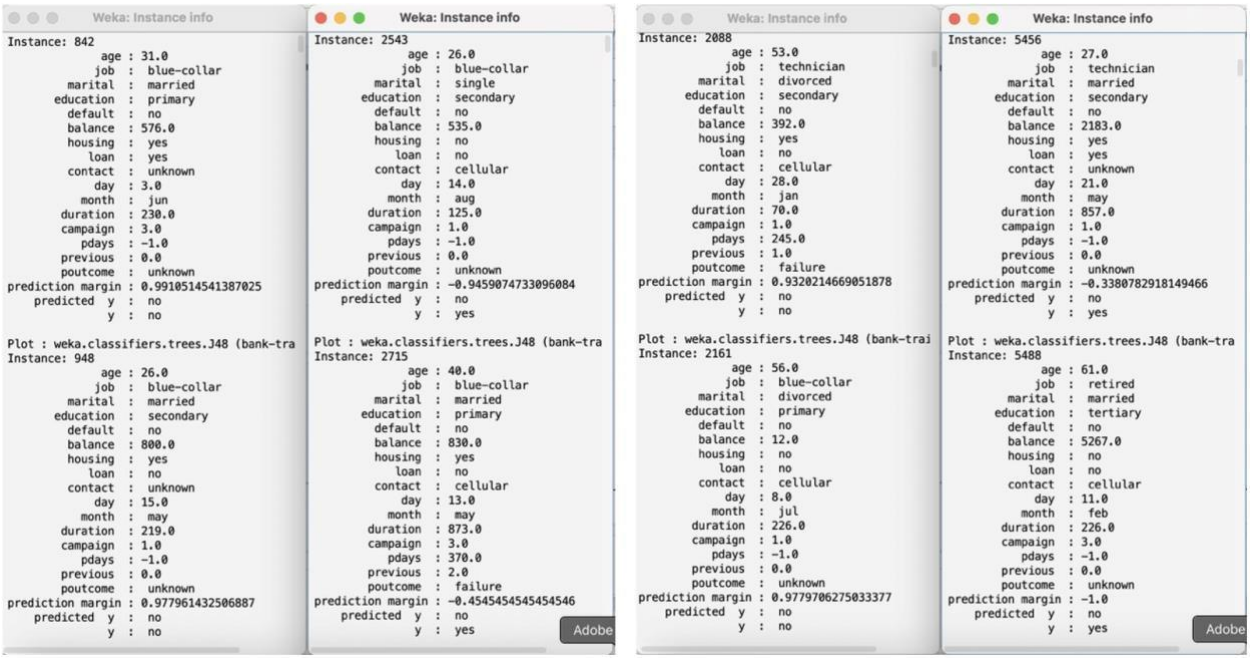

Figure 5: Instance Comparison between Actual No's and False Yes's

By clicking the clusters in the graph, I got the instances. It appears that both groups typically "no" loan, Default are always "no", p-outcome is typically "unknown". Connecting to the context, it might be important to know whether the client currently has a loan or not to determine if they have extra money to subscribe. But the outcome of the previous campaign and credit in default seems overall similar across groups. Hence, based on the vertical analysis, I suggest removing default and/or poutcome. Now switch to horizontal analysis.

Error Analysis – Horizontal Analysis

For Horizontal Comparison, I focus on the difference between actual yes's and yes marks as no's. Because by removing or modifying the difference in feature it can help banks identify clients that are more likely to subscribe.

Predicted Actual	No	Yes
Yes	1785	1543

Table 4: Actual Subscribed vs. Subscribe but Mark as Not Subscribed

And here are the instance comparison:

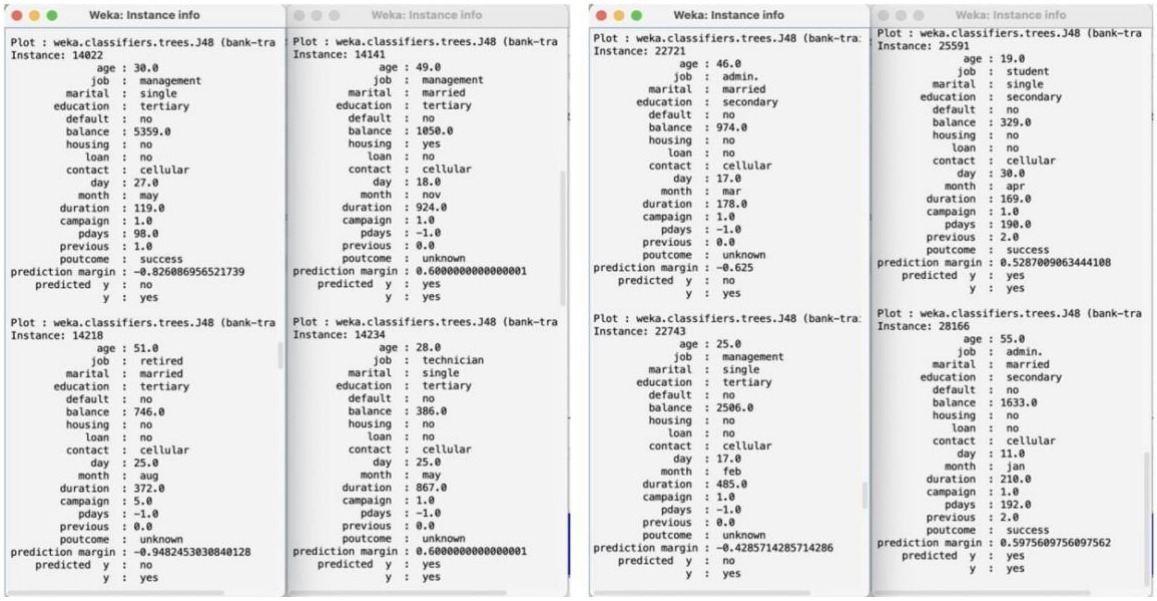

Figure 6: Instance Comparison of False No's vs. Actual Yes's

By comparison, I notice the balance the client has differs from yes marked as no and actual yes. It seems like a client that subscribed but marked as no subscriber has a higher balance than correctly classified subscribers. Also, the month is greatly varied from each group.

Therefore, I suggest removing balance and/or month.

Error Analysis – Feature Selection

Based on the analysis, I should try to remove default, month, poutcome, and balance. Not all at once but try different orders and combinations. In the training-train data, I removed Default first and ran J48 again. The performance went down by 0.01.

```
=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       25690               90.1973 %
Incorrectly Classified Instances      2792                9.8027 %
Kappa statistic                          0.4708
Mean absolute error                      0.1284
Root mean squared error                  0.2804
Relative absolute error                 62.2175 %
Root relative squared error             87.2881 %
Total Number of Instances            28482
```

Figure 7: Training data–remove default–J48

I removed p-outcome and the performance went down by 1 percent to 89.6%, and additional removal of month and balance resulted at 89% with decreased performance in decimal places (89.6->89.09->89.04). Hence it appears that removing p-outcome isn't the right decision. It is a more important factor than the other features I was planning to remove. Removing balance results in performance increase by 0.1 percent. Removing the month alone, however, led to 1 percent decrease in performance. Therefore, before using the supplied test, the decision is to remove Balance and default. The model's performance based on the training-train data is 90.29%

```
Correctly Classified Instances        25719               90.2991 %
Incorrectly Classified Instances       2763                9.7009 %
Kappa statistic                           0.4742
Mean absolute error                       0.1284
Root mean squared error                   0.276
Relative absolute error                  62.2002 %
Root relative squared error              85.9198 %
Total Number of Instances             28482
```

Figure 8: Training Data – Removing Balance and Default

```
Correctly Classified Instances         2854               90.1738 %
Incorrectly Classified Instances        311                9.8262 %
Kappa statistic                           0.4812
Mean absolute error                       0.1276
Root mean squared error                   0.2782
Relative absolute error                  61.2467 %
Root relative squared error              85.79   %
Total Number of Instances              3165
```

Figure 9: Supplied Test: Removing Balance and Default

```
Correctly Classified Instances         2852               90.1106 %
Incorrectly Classified Instances        313                9.8894 %
Kappa statistic                           0.4725
Mean absolute error                       0.1275
Root mean squared error                   0.2819
Relative absolute error                  61.1955 %
Root relative squared error              86.9328 %
Total Number of Instances              3165
```

Figure 10: Supplied Test: Original no removal data

Figure 9 shows the performance of the supplied test of the 10% training dataset on the 90% training dataset with balance and default removed. The accuracy is 90.17%. Figure 10 shows the performance of the supplied test with 10% original training data on the 90% original training data. The performance is 90.11%. Therefore, the model after removing default and

balance actually performs better, implying balance is a confusing feature for the model, and default is an almost irrelevant feature that can be removed.

Parameter Tuning

Another way to improve the general performance of the model is through parameter tuning. In J48 decision tree model, the confidence factor is a parameter used for pruning, with smaller number incurring more pruning; the batch size is the preferred number of instances to process if batch prediction is being performed. Pruning is a great way to avoid overfitting and to cut unnecessary branches off from the tree. Batch size may be useful in terms of information provided each time for the tree to complete itself.

To conduct parameter tuning, I split the training data into ten equal sized folds (7 folds of 3165 instances and 3 folds with 3164). Five folds will be used as training folds, and five folds will be used as testing folds.

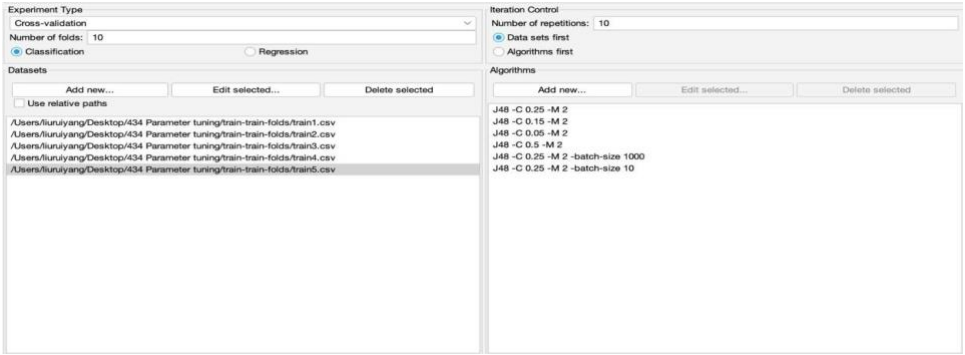

Figure 11: Parameter Tuning set up in Experiment

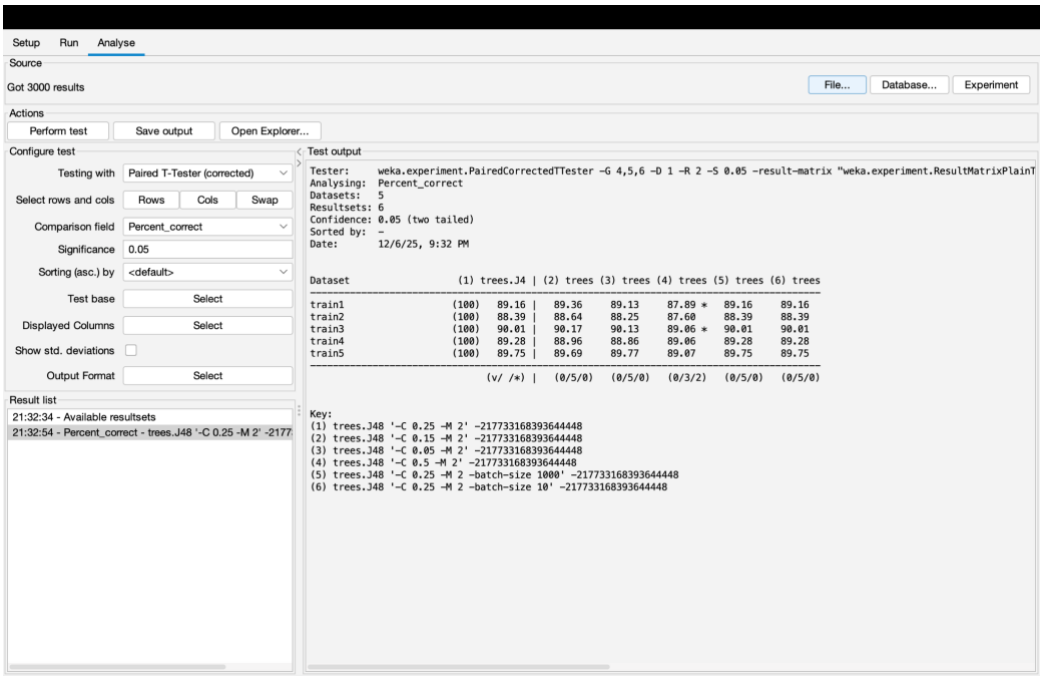

Figure 12: Performance of Training Data with Different Confidence Factor or Batch Size

The first step is to run different parameter values on each fold using the Weka experimenter mode. Figure 11 shows the basic setup of the experiment, with the confidence factor tuned to five different values: 0.25(default), 0.15, 0.05, 0.5; and two separate models where the batch size is set to 1000 and 10 while the confidence factor is set to default 0.25. Figure 12 shows the result of the model performance on each fold. Change in batch size has no effect on the performance of the model, and it seems confidence factor at 0.15 has a better performance on average than other confidence factor values.

=TTEST(A3:A7,B3:B7,2,1)

A	B	C	D
Confidence factor = 0.15			
Train	Test		T-test
0.8936	0.8985		0.786730887
0.8864	0.897		
0.9017	0.8945		
0.8896	0.8825		
0.8969	0.9008		

Figure 13: T-test on train-train and train-test folds performance

Using 0.15 as the new confidence factor, I then apply the supplied test using the testing folds separated from training data to test on the training folds. A T-test is used to test significance of the parameter tuning with null hypothesis being there is no significant difference between tuning and no tuning. The p-value of the T-test is 0.78 and failure to reject the null hypothesis, meaning there is no significant evidence that the parameter tuning confidence factor is better than the default model.

Final Evaluation

Based on the previous analysis, the final classification model selected is the J48 model, with the default setting of confidence factor equal to 0.25 and nothing changed. The features that are removed from the dataset are the balance and default variables. Finally, the testing dataset can be used through supplied tests to the training dataset.

```
=== Summary ===

Correctly Classified Instances        4117               91.0438 %
Incorrectly Classified Instances       405                8.9562 %
Kappa statistic                          0.5117
Mean absolute error                      0.1218
Root mean squared error                  0.2657
Relative absolute error                 60.4425 %
Root relative squared error             84.8663 %
Total Number of Instances             4522

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.959    0.480    0.942      0.959    0.950      0.514    0.862     0.966      no
                 0.520    0.041    0.609      0.520    0.561      0.514    0.862     0.489      yes
Weighted Avg.    0.910    0.432    0.905      0.910    0.907      0.514    0.862     0.913

=== Confusion Matrix ===

    a    b   <-- classified as
 3858  166 |    a =  no
  239  259 |    b =  yes
```

Figure 14: Model Performance on the Final Testing Data (Balance and Default Removed)

```
Correctly Classified Instances        4121               91.1322 %
Incorrectly Classified Instances       401                8.8678 %
Kappa statistic                          0.5138
Mean absolute error                      0.1215
Root mean squared error                  0.2696
Relative absolute error                 60.3239 %
Root relative squared error             86.0867 %
Total Number of Instances             4522

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.960    0.482    0.942      0.960    0.951      0.516    0.850     0.962      no
                 0.518    0.040    0.616      0.518    0.563      0.516    0.850     0.472      yes
Weighted Avg.    0.911    0.433    0.906      0.911    0.908      0.516    0.850     0.908

=== Confusion Matrix ===

    a    b   <-- classified as
 3863  161 |    a =  no
  240  258 |    b =  yes
```

Figure 15: Model Performance with Original Train and Testing Data For comparison, I also applied the supplied test using the original dataset without removing balance and default. Figure 14 shows the performance of feature selected data is 91.04 percent. Figure 15 shows the performance for the original dataset using J48 is 91.13 percent. The result of the feature selected dataset didn't perform better than the original dataset.

Conclusion

The final evaluation implies that using feature selection, specifically removing the default variable of whether the client has credit and average yearly balance in euros, didn't improve the performance of predicting whether a client will subscribe to a term deposit.

From this project, I have acquired a better understanding about the general process of using machine learning models to predict the class variables that I or the client is interested in. I also learned the most intensive work happens in preparing the data and fine tuning the model using error analysis and parameter tuning. A data-driven approach with machine learning is not as easy as simply loading in the dataset and run the model, it is a careful process of understanding dataset, figure out the class variable interested, clean the data so it's usable, explore different baseline models to find the best model applicable, examine the errors the model makes and try to fix it using horizontal and vertical analysis and more techniques, change parameter of the model and see if it does better.

Two of the interesting lessons I learned is firstly, despite the model seems to be improving in the training process, it might be overfitting or simply the dataset is more clustered in a high dimensional space. Once applied to the testing dataset the eventual result might be lower than what you expected, which is exactly what happened in my model. Secondly, the data cleaning process, including storing data in a different form, is crucial before feeding data into the model. Initially I didn't change the attribute format description in the .arff files, and Weka couldn't run the supplied test because it was unable to read two different nominal orders from two dataset with the same variable. As I fixed that problem, the model was able to produce the correct result.

A future approach I will try to improve is the idea of combining two or more features to investigate stronger association between the ratio and the class variable. Many times the true association lies within the combination of some seemingly unrelated variables.

Works Cited

Mollaev, Dzhambulat, et al. "*Multimodal Banking Dataset: Understanding Client needs through event sequences.*" Proceedings of the 34th ACM International Conference on Information and Knowledge Management, 10 Nov. 2025, pp. 6476–6480, https://doi.org/10.1145/3746252.3761635.

Moro, S., P. Rita, and P. Cortez. "*A data-driven approach to predict the success of bank telemarketing*" Elsevier, 2014, https://doi.org/10.1016/j.dss.2014.03.001

Moro, S., P. Rita, and P. Cortez. "*Bank Marketing.*" UCI Machine Learning Repository, 2014, https://doi.org/10.24432/C5K306.

[Moro et al., 2011] S. Moro, R. Laureano and P. Cortez. Using Data Mining for Bank Direct Marketing: An Application of the CRISP-DM Methodology.

In P. Novais et al. (Eds.), Proceedings of the European Simulation and Modelling Conference - ESM'2011, pp. 117-121, Guimarães, Portugal, October, 2011. EUROSIS.

Available at: [pdf] http://hdl.handle.net/1822/14838

[bib] http://www3.dsi.uminho.pt/pcortez/bib/2011-esm-1.txt